REOP 2021 - Session 1 (Problems and graphs)

Guillaume Dalle

September 22, 2021

Contents

1 Introduction **1**
1.1 About me . . . 1
1.2 Resources . . . 1
1.3 Rules . . . 1

2 General framework of OR **2**
2.1 Problems . . . 2
2.2 Algorithms and heuristics . . . 2

3 Graphs **2**
3.1 Vocabulary . . . 2
3.2 Paths . . . 3
3.3 Types of graphs . . . 3
3.4 Zoo of graph problems . . . 3

4 MILP modeling **4**

1 Introduction

1.1 About me

Hi, I'm Guillaume DALLE, a PhD student in applied mathematics at CERMICS. I work with SNCF on predicting and preventing traffic disruptions using tools from machine learning and operations research.

The easiest way to contact me is my academic email guillaume.dalle@enpc.fr. I will answer quickly but never outside of office hours, in order to preserve your personal life and mine. You can also come visit me in room B210 of the Coriolis building at École des Ponts, just remember to schedule it with me beforehand.

1.2 Resources

- Course website on Educnet (https://educnet.enpc.fr/course/view.php?id=1215)
- My website (https://gdalle.github.io/reop/)
- The Teams workspace (see email from Axel Parmentier)

1.3 Rules

- Be respectful towards your classmates (and teacher).
- Always ask me if you have any problems with the class or with your studies in general. Honesty will never be punished, and I may even be able to help.
- Follow Covid sanitary guidelines. Masks on the mouth and nose at all times.

- Try to arrive on time, send an email if you can't.
- Language should not be an issue.

2 General framework of OR

2.1 Problems

Problem $\mathcal{P}$ = family of inputs + question to answer:

- Decision problem: answer is either `yes` or `no`
- Optimization problem: find the best candidate among a family

An optimization problem is often formulated as

$$\min_x c(x) \quad \text{s.t.} \quad x \in \mathcal{X} \tag{P}$$

- "s.t." means "subject to"
- x is the decision variable
- $\mathcal{X}$ is the set of feasible solutions
- $x \in \mathcal{X}$ is the set of constraints
- $c(x)$ is the criterion / objective / cost function

Solving the problem means finding

- the optimal value $\text{val}(P)$
- if possible, an optimal solution $x^* \in \text{argmin}\{c(x) : x \in \mathcal{X}\}$

Instance $\mathcal{I}$ = one particular input with its numerical values.

2.2 Algorithms and heuristics

Algorithm $\mathcal{A}$ = sequence of elementary operations that can be implemented on a computer.

Time complexity $f(n)$ of an algorithm $\mathcal{A}$ = number of elementary operations necessary with input of size n. Polynomial algorithms are usually considered efficient, while exponential algorithms scare many researchers (not us).

There are several types of optimization algorithms:

- Exact algorithms: yield an optimal solution
- Approximation algorithms: yield a solution with bounded sub-optimality
- Heuristics: yield a solution with no guarantee at all

Local descent enables us to explore large solution spaces with small movements. Typical algorithm: given the current solution x_k

1. Compute and explore its neighborhood $\mathcal{N}(x_k)$
2. Pick a next solution $x_{k+1} \in \mathcal{N}(x_k)$ such that $c(x_k) < c(x_{k+1})$

How to evaluate the quality of a solution without a certificate of optimality? Compare its value $c(x)$ with a lower bound $\ell \leq \text{val}(P) \leq c(x)$

3 Graphs

3.1 Vocabulary

A graph $G = (V, E)$ is composed of:

- a set V of vertices, often called u or v, with $|V| = n$

- a set E of edges, often denoted by $e = (u, v)$, with $|E| = m$

A subgraph H is a couple $(V' \subset V, E' \subset E)$.

The degree of a vertex is the number of incidence edges

The adjacency matrix $A = (A_{u,v}) \in \{0, 1\}^{n \times n}$ tells us whether two vertices are linked by an edge

3.2 Paths

A path is a sequence of nodes linked by edges. Types of paths:

- simple: no edge is crossed twice
- elementary: no vertex visited twice
- cycle: start vertex = end vertex
- eulerian: crosses all edges once
- hamiltonian: visits all vertices once

3.3 Types of graphs

- undirected: edges are not ordered $(u, v) = \{u, v\}$
- directed: edges are ordered $(u, v) = u \to v$ (we use the terminology "nodes" and "arcs")
- simple: no duplicate edges or self loops
- complete: simple with all possible edges
- bipartite: split in two parts with all edges in the middle
- connected: all couples of vertices have a path between them
- eulerian / hamiltonian: contains a eulerian/hamiltonian cycle
- forest: contains no cycle
- tree: connected and contains no cycle

A graph is eulerian iff all its vertices have even degree (i.e. an even number of incident edges). Proof: construct the cycle.

3.4 Zoo of graph problems

Colorings and cliques

A coloring is a function $c : V \to \mathbb{N}$ such that two adjacent vertices do not share the same color: if $(u, v) \in E$, $c(u) \neq c(v)$. The minimum number of colors needed to color a graph is its chromatic number $\chi(G)$.

A clique is a complete subgraph. The maximum cardinality of a clique is denoted by $\omega(G)$.

We have $\omega(G) \leq \chi(G)$.

Matchings and covers

A matching is a set of disjoint edges (maximum cadinality $\nu(G)$).

A vertex cover is a set of vertices that covers (has a member belonging to) every edge (minimum cardinality $\tau(G)$).

We have $\nu(G) \leq \tau(G)$.

Stable sets and edge covers

A stable set is a set of vertices such that no two of them are linked to each other (maximum cardinality $\alpha(G)$).

An edge cover is a set of edges that covers (has a member incident to) every vertex (minimum cardinality $\rho(G)$).

We have $\alpha(G) \leq \rho(G)$.

4 MILP modeling

A Linear Program is an optimization problem with a linear objective and linear constraints:

$$\min_{x \in \mathbb{R}^n} c^\top x \quad \text{s.t.} \quad Ax \leq b \tag{LP}$$

A Mixed Integer Linear Program is a LP where some of the variables are constrained to be integers

$$\min_{x \in \mathbb{Z}^p \times \mathbb{R}^{n-p}} c^\top x \quad \text{s.t.} \quad Ax \leq b \tag{MILP}$$

In theory, solving a MILP is hard. However:

- they are very useful to model lots of real-life problems
- there are practically efficient solvers that can handle millions of variables if the problem has a certain structure

---

Homework

- Fill the introduction form on Wooclap if you haven't already
- Exercises: 3.2 (Eulerian graphs), 3.3 (adjacency matrices) and 3.10 (stable sets and vertex covers)

References